

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«Национальный исследовательский университет ИТМО»
(Университет ИТМО)

Факультет **Прикладной информатики**

Направление подготовки **09.03.03 Прикладная информатика**

Образовательная программа **Мобильные и сетевые технологии**

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ №6
по дисциплине “Проектирование и реализация баз данных”

Обучающийся: Захаркин Богдан Владимирович, К3440

Преподаватель: Говорова Марина Михайловна

Санкт-Петербург
2026

Цель работы: овладеть практическими навыками работы с CRUD-операциями, с вложенными объектами в коллекции базы данных MongoDB, агрегации и изменения данных, со ссылками и индексами в базе данных MongoDB.

Выполнение:

1 Лабораторная работа 6.1

1.1 Версия

```
root@spb-3-vm-ucia:/etc/apt/sources.list.d# mongod --version
db version v7.0.28
Build Info: {
  "version": "7.0.28",
  "gitVersion": "cfc346c59d2cc3dbc903507fc642e22e3097f362",
  "opensslVersion": "OpenSSL 3.0.13 30 Jan 2024",
  "modules": [],
  "allocator": "tcmalloc",
  "environment": {
    "distmod": "ubuntu2204",
    "distarch": "x86_64",
    "target_arch": "x86_64"
  }
}
```

1.2 Запуск клиента

```
root@spb-3-vm-ucia:~# mongosh
Current Mongosh Log ID: 696a1e5c94c608074e8ce5af
Connecting to:      mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&app
Name=mongosh+2.6.0
Using MongoDB:      7.0.28
Using Mongosh:      2.6.0

For mongosh info see: https://www.mongodb.com/docs/mongosh-shell/

-----
The server generated these startup warnings when booting
2026-01-16T11:01:58.000+00:00: Using the XFS filesystem is strongly recommended with the WiredTiger storage engine. See http://dochub.mongodb.org/core/prodnotes-filesystem
2026-01-16T11:01:58.783+00:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
-----
test>
```

1.3 Выполнить методы

— db.help()

```
test> db.help()

Database Class:

getMongo           Returns the current database connection
getName           Returns the name of the DB
getCollectionNames Returns an array containing the names of all collections in
the current database.
getCollectionInfos Returns an array of documents with collection information,
i.e. collection name and options, for the current database.
runCommand         Runs an arbitrary command on the database.
adminCommand       Runs an arbitrary command against the admin database.
aggregate          Runs a specified admin/diagnostic pipeline which does not require an underlying collection.
getSiblingDB       Returns another database without modifying the db variable in the shell environment.
getCollection       Returns a collection or a view object that is functionally equivalent to using the db.<collectionName>.
dropDatabase       Removes the current database, deleting the associated data files.
createUser         Creates a new user for the database on which the method is run. db.createUser() returns a duplicate user error if the user already exists on the database.
updateUser         Updates the user's profile on the database on which you run the method. An update to a field completely replaces the previous field's values. This includes updates to the user's roles array.
changeUserPassword Updates a user's password. Run the method in the database where the user is defined, i.e. the database you created the user.
logout            Ends the current authentication session. This function has no effect if the current session is not authenticated.
dropUser          Removes the user from the current database.
dropAllUsers       Removes all users from the current database.
auth             Allows a user to authenticate to the database from within the shell.
grantRolesToUser   Grants additional roles to a user.
revokeRolesFromUser Removes a one or more roles from a user on the current database.
getUser           Returns user information for a specified user. Run this method on the user's database. The user must exist on the database on which the method runs.
getUsers          Returns information for all the users in the database.
createCollection   Create new collection
```

– db.help

```
test> db.help

Database Class:

  getMongo           Returns the current database connection
  getName            Returns the name of the DB
  getCollectionNames Returns an array containing the names of all collections in
the current database.
  getCollectionInfos Returns an array of documents with collection information,
i.e. collection name and options, for the current database.
  runCommand         Runs an arbitrary command on the database.
  adminCommand       Runs an arbitrary command against the admin database.
  aggregate          Runs a specified admin/diagnostic pipeline which does not r
quire an underlying collection.
  getSiblingDB       Returns another database without modifying the db variable
in the shell environment.
  getCollection       Returns a collection or a view object that is functionally
equivalent to using the db.<collectionName>.
  dropDatabase       Removes the current database, deleting the associated data
files.
  createUser         Creates a new user for the database on which the method is
run. db.createUser() returns a duplicate user error if the user already exists on the database.
  updateUser         Updates the user's profile on the database on which you run
the method. An update to a field completely replaces the previous field's values. This includes updates t
o the user's roles array.
  changeUserPassword Updates a user's password. Run the method in the database w
here the user is defined, i.e. the database you created the user.
  logout             Ends the current authentication session. This function has
no effect if the current session is not authenticated.
  dropUser           Removes the user from the current database.
  dropAllUsers       Removes all users from the current database.
  auth              Allows a user to authenticate to the database from within t
he shell.
  grantRolesToUser   Grants additional roles to a user.
  revokeRolesFromUser Removes a one or more roles from a user on the current data
base.
  getUser            Returns user information for a specified user. Run this met
hod on the user's database. The user must exist on the database on which the method runs.
```

– db.stats()

```
test> db.stats()
{
  db: 'test',
  collections: Long('0'),
  views: Long('0'),
  objects: Long('0'),
  avgObjSize: 0,
  dataSize: 0,
  storageSize: 0,
  indexes: Long('0'),
  indexSize: 0,
  totalSize: 0,
  scaleFactor: Long('1'),
  fsUsedSize: 0,
  fsTotalSize: 0,
  ok: 1
}
test>
```

1.4 Создайте БД learn

```
test> use learn
switched to db learn
```

1.5 Получите список доступных БД

Таблица learn появится после вставки документа.

```
learn> show dbs
admin    40.00 KiB
config   12.00 KiB
local    40.00 KiB
learn> 
```

1.6 Создайте коллекцию unicorns и просмотрите список текущих коллекций

```
learn> show collections

learn> db.unicorns.insert({name: 'Aurora', gender: 'f', weight: 450})
DeprecationWarning: Collection.insert() is deprecated. Use insertOne, insertMany, or bulkWrite.
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('696a1f4d94c608074e8ce5b0') }
}
learn> show collections
unicorns
learn>
```

1.7 Переименуйте коллекцию unicorns

```
learn> db.unicorns.renameCollection('ednorok')
{ ok: 1 }
learn> show collections
ednorok
learn>
```

1.8 Посмотрите статистику коллекции

```
learn> db.ednorok.stats()
{
  ok: 1,
  capped: false,
  wiredTiger: {
    metadata: { formatVersion: 1 },
    creationString: 'access_pattern_hint=none,allocation_size=4KB,app_metadata=(formatVersion=1),assert=(commit_timestamp=none,durable_timestamp=none,read_timestamp=none,write_timestamp=on),block_allocation=best,block_compressor=snappy,cache_resident=false,checksum=on,colgroups=,collator=,columns=,dictionary=0,encryption=(keyid=,name=),exclusive=false,extractor=,format=btree,huffman_key=huffman_value,ignore_in_memory_cache_size=false,immutable=false,import=(compare_timestamp=oldest_timestamp,enabled=false,file_metadata=,metadata_file=,repair=false),internal_item_max=0,internal_key_max=0,internal_key_truncate=true,internal_page_max=4KB,key_format=q,key_gap=10,leaf_item_max=0,leaf_key_max=0,leaf_page_max=32KB,leaf_value_max=64MB,log=(enabled=true),lsm=(auto_throttle=true,bloom=true,bloom_bit_count=16,bloom_config=,bloom_hash_count=8,bloom_oldest=false,chunk_count_limit=0,chunk_max=50B,chunk_size=10MB,merge_custom=(prefix,start_generation=0,suffix=),merge_max=15,merge_min=0),memory_page_image_max=0,memory_page_max=10n,os_cache_dirty_max=0,os_cache_max=0,prefix_compression=false,prefix_compression_min=4,source=,split_deepen_min_child=0,split_deepen_per_child=0,split_pct=98,tiered_storage=(auth_token=,bucket=,bucket_prefix=,cache_directory=,local_retention=300,name=,object_target_size=0),type=file,uuid_format=,verbose=[write_timestamp],write_timestamp_usage=none',
    uri: 'statistics:table:collection-9--5145368635348178102',
    LSM: {
      'bloom filter false positives': 0,
      'bloom filter hits': 0,
      'bloom filter misses': 0,
      'bloom filter pages evicted from cache': 0,
      'bloom filter pages read into cache': 0,
      'bloom filters in the LSM tree': 0,
      'chunks in the LSM tree': 0,
      'highest merge generation in the LSM tree': 0,
      'queries that could have benefited from a Bloom filter that did not exist': 0,
      'sleep for LSM checkpoint throttle': 0,
      'sleep for LSM merge throttle': 0,
      'total size of bloom filters': 0
    },
    autocommit: {
      'retries for readonly operations': 0,
      'retries for update operations': 0
    },
    'block-manager': {
      'allocations requiring file extension': 4,
      'blocks allocated': 4,
      'blocks freed': 0,
      'checkpoint size': 4096,
      'file allocation unit size': 4096,
      'file bytes available for reuse': 0,
      'file magic number': 120897,
      'file major version number': 1,
      'file size in bytes': 20480,
      'minor version number': 0
    },
    btree: {
      'btree checkpoint generation': 23,
      'btree clean tree checkpoint expiration time': Long('9223372036854775807'),
      'btree compact pages reviewed': 0,
      'btree compact pages rewritten': 0,
      'btree compact pages skipped': 0,
      'btree number of pages reconciled during checkpoint': 2,
      'btree skipped by compaction as process would not reduce size': 0,
      'column-store fixed-size leaf pages': 0,
      'column-store fixed-size time windows': 0,
      'column-store internal pages': 0,
      'column-store variable-size RLE encoded values': 0,

```

1.9 Удалите коллекцию

```
learn> show collections
ednorok
learn> db.ednorok.drop()
true
learn> show collections
learn>
```

1.10 Удалите БД learn

```
learn> db.dropDatabase()  
{ ok: 1, dropped: 'learn' }  
learn> show dbs  
admin    40.00 KiB  
config  108.00 KiB  
local    40.00 KiB  
learn>
```

2 Лабораторная работа 6.2 раздел 2

2.1 Задание 2.1.1

- *Создайте базу данных learn.*
- *Заполните коллекцию единорогов unicorns:*

```
db.unicorns.insert({name: 'Horny', loves: ['carrot','papaya'], weight: 600, gender: 'm', vampires: 63});
db.unicorns.insert({name: 'Aurora', loves: ['carrot', 'grape'], weight: 450, gender: 'f', vampires: 43});
db.unicorns.insert({name: 'Unicrom', loves: ['energon', 'redbull'], weight: 984, gender: 'm', vampires: 182});
db.unicorns.insert({name: 'Rooooooodles', loves: ['apple'], weight: 575, gender: 'm', vampires: 99});
db.unicorns.insert({name: 'Solnara', loves:['apple', 'carrot', 'chocolate'], weight:550, gender:'f', vampires:80});
db.unicorns.insert({name:'Ayna', loves: ['strawberry', 'lemon'], weight: 733, gender: 'f', vampires: 40});
db.unicorns.insert({name:'Kenny', loves: ['grape', 'lemon'], weight: 690, gender: 'm', vampires: 39});
db.unicorns.insert({name: 'Raleigh', loves: ['apple', 'sugar'], weight: 421, gender: 'm', vampires: 2});
db.unicorns.insert({name: 'Leia', loves: ['apple', 'watermelon'], weight: 601, gender: 'f', vampires: 33});
db.unicorns.insert({name: 'Pilot', loves: ['apple', 'watermelon'], weight: 650, gender: 'm', vampires: 54});
db.unicorns.insert({name: 'Nimue', loves: ['grape', 'carrot'], weight: 540, gender: 'f'});
```

Решили объединить объединить оба этих пункта в один скриншот. На удивление можно прямо скопировать и вставить код, и он вставит все указанные документы, но вывод отобразится только у последнего, но вставились все, дальше это будет видно.

```
test> use learn
switched to db learn
learn> db.unicorns.insert({name: 'Horny', loves: ['carrot','papaya'], weight: 600, gender: 'm', vampires: 63});
... db.unicorns.insert({name: 'Aurora', loves: ['carrot', 'grape'], weight: 450, gender: 'f', vampires: 43});
... db.unicorns.insert({name: 'Unicrom', loves: ['energon', 'redbull'], weight: 984, gender: 'm', vampires: 182});
... db.unicorns.insert({name: 'Rooooooodles', loves: ['apple'], weight: 575, gender: 'm', vampires: 99});
... db.unicorns.insert({name: 'Solnara', loves:['apple', 'carrot', 'chocolate'], weight:550, gender:'f', vampires:80});
... db.unicorns.insert({name:'Ayna', loves: ['strawberry', 'lemon'], weight: 733, gender: 'f', vampires: 40});
... db.unicorns.insert({name:'Kenny', loves: ['grape', 'lemon'], weight: 690, gender: 'm', vampires: 39});
... db.unicorns.insert({name: 'Raleigh', loves: ['apple', 'sugar'], weight: 421, gender: 'm', vampires: 2});
... db.unicorns.insert({name: 'Leia', loves: ['apple', 'watermelon'], weight: 601, gender: 'f', vampires: 33});
... db.unicorns.insert({name: 'Pilot', loves: ['apple', 'watermelon'], weight: 650, gender: 'm', vampires: 54});
... db.unicorns.insert({name: 'Nimue', loves: ['grape', 'carrot'], weight: 540, gender: 'f'});
...
DeprecationWarning: Collection.insert() is deprecated. Use insertOne, insertMany, or bulkWrite.
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('696a2d5fc6472676be8ce5ba') }
}
```

— *Используя второй способ, вставьте в коллекцию единорогов документ:*

```
{name: 'Dunx', loves: ['grape', 'watermelon'], weight: 704, gender: 'm', vampires: 165}
```

```
learn> document = ({name: 'Dunx', loves: ['grape', 'watermelon'], weight: 704, gender: 'm', vampires: 165})
{
  name: 'Dunx',
  loves: [ 'grape', 'watermelon' ],
  weight: 704,
  gender: 'm',
  vampires: 165
}
learn> db.unicorns.insert(document)
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('696a2df5c6472676be8ce5bb') }
}
```

— *Проверьте содержимое коллекции с помощью метода find.*

```
learn> db.unicorns.find()
[
  {
    _id: ObjectId('696a2d5fc6472676be8ce5b0'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('696a2d5fc6472676be8ce5b1'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('696a2d5fc6472676be8ce5b2'),
    name: 'Unicrom',
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 182
  },
  {
    _id: ObjectId('696a2d5fc6472676be8ce5b3'),
    name: 'Rooooooodles',
    loves: [ 'apple' ],
    weight: 575,
    gender: 'm',
    vampires: 99
  },
  {
    _id: ObjectId('696a2d5fc6472676be8ce5b4'),
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    _id: ObjectId('696a2d5fc6472676be8ce5b5'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 733,
    gender: 'f',
    vampires: 40
  },
  {
    _id: ObjectId('696a2d5fc6472676be8ce5b6'),
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  },
  {
    _id: ObjectId('696a2d5fc6472676be8ce5b5'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 733,
    gender: 'f',
    vampires: 40
  },
  {
    _id: ObjectId('696a2d5fc6472676be8ce5b7'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  },
  {
    _id: ObjectId('696a2d5fc6472676be8ce5b8'),
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  },
  {
    _id: ObjectId('696a2d5fc6472676be8ce5b9'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],
    weight: 650,
    gender: 'm',
    vampires: 54
  },
  {
    _id: ObjectId('696a2d5fc6472676be8ce5ba'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  },
  {
    _id: ObjectId('696a2df5c6472676be8ce5bb'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  }
]
learn>
```


2.2 Задание 2.2.1

- *Сформируйте запрос для вывода списков самцов единорогов.*

Отсортируйте списки по имени.

```
learn> db.unicorns.find({gender:'m'}).sort({name:1})
[
  {
    _id: ObjectId('696a2df5c6472676be8ce5bb'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  },
  {
    _id: ObjectId('696a2d5fc6472676be8ce5b0'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('696a2d5fc6472676be8ce5b6'),
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  },
  {
    _id: ObjectId('696a2d5fc6472676be8ce5b9'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],
    weight: 650,
    gender: 'm',
    vampires: 54
  },
  {
    _id: ObjectId('696a2d5fc6472676be8ce5b7'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  },
  {
    _id: ObjectId('696a2d5fc6472676be8ce5b3'),
    name: 'Rooooooodles',
    loves: [ 'apple' ],
    weight: 575,
    gender: 'm',
    vampires: 99
  },
  {
    _id: ObjectId('696a2d5fc6472676be8ce5b2'),
    name: 'Unicrom',
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 182
  }
]
learn>
```

– Сформируйте запрос для вывода списков самок единорогов. Ограничьте список самок первыми тремя особями. Отсортируйте списки по имени.

```
learn> db.unicorns.find({gender:'f'}).limit(3).sort({name:1})
[
  {
    _id: ObjectId('696a2d5fc6472676be8ce5b1'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('696a2d5fc6472676be8ce5b5'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 733,
    gender: 'f',
    vampires: 40
  },
  {
    _id: ObjectId('696a2d5fc6472676be8ce5b8'),
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  }
]
learn> 
```

– Найдите всех самок, которые любят carrot. Ограничьте этот список первой особью с помощью функций `findOne` и `limit`.

```
learn> db.unicorns.find({gender:'f', loves:'carrot'}).limit(1)
[
  {
    _id: ObjectId('696a2d5fc6472676be8ce5b1'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  }
]
learn> db.unicorns.findOne({gender:'f', loves:'carrot'})
{
  _id: ObjectId('696a2d5fc6472676be8ce5b1'),
  name: 'Aurora',
  loves: [ 'carrot', 'grape' ],
  weight: 450,
  gender: 'f',
  vampires: 43
}
learn> 
```

2.3 Задание 2.2.2

Модифицируйте запрос для вывода списков самцов единорогов, исключив из результата информацию о предпочтениях и поле.

```
learn> db.unicorns.find({gender:'m'},{loves:0,gender:0}).sort({name:1})
[
  {
    _id: ObjectId('696a2df5c6472676be8ce5bb'),
    name: 'Dunx',
    weight: 704,
    vampires: 165
  },
  {
    _id: ObjectId('696a2d5fc6472676be8ce5b0'),
    name: 'Horny',
    weight: 600,
    vampires: 63
  },
  {
    _id: ObjectId('696a2d5fc6472676be8ce5b6'),
    name: 'Kenny',
    weight: 690,
    vampires: 39
  },
  {
    _id: ObjectId('696a2d5fc6472676be8ce5b9'),
    name: 'Pilot',
    weight: 650,
    vampires: 54
  },
  {
    _id: ObjectId('696a2d5fc6472676be8ce5b7'),
    name: 'Raleigh',
    weight: 421,
    vampires: 2
  },
  {
    _id: ObjectId('696a2d5fc6472676be8ce5b3'),
    name: 'Rooooooodles',
    weight: 575,
    vampires: 99
  },
  {
    _id: ObjectId('696a2d5fc6472676be8ce5b2'),
    name: 'Unicrom',
    weight: 984,
    vampires: 182
  }
]
learn>
```

2.4 Задание 2.2.3

Вывести список единорогов в обратном порядке добавления.

```
learn> db.unicorns.find().sort({$natural:-1})
[
  {
    _id: ObjectId('696a2df5c6472676be8ce5bb'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  },
  {
    _id: ObjectId('696a2d5fc6472676be8ce5ba'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  },
  {
    _id: ObjectId('696a2d5fc6472676be8ce5b9'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],
    weight: 650,
    gender: 'm',
    vampires: 54
  },
  {
    _id: ObjectId('696a2d5fc6472676be8ce5b8'),
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  },
  {
    _id: ObjectId('696a2d5fc6472676be8ce5b7'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  },
  {
    _id: ObjectId('696a2d5fc6472676be8ce5b6'),
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  },
  {
    _id: ObjectId('696a2d5fc6472676be8ce5b5'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 733,
    gender: 'f',
    vampires: 40
  },
  {
    _id: ObjectId('696a2d5fc6472676be8ce5b4'),
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    _id: ObjectId('696a2d5fc6472676be8ce5b3'),
    name: 'Rooooooodles',
    loves: [ 'apple' ],
    weight: 575
  }
]
```

2.5 Задание 2.2.4

Вывести список единорогов с названием первого любимого предпочтения, исключив идентификатор.

```
learn> db.unicorns.find({}, {loves: {$slice: 1}, _id: 0})
[
  {
    name: 'Horny',
    loves: [ 'carrot' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    name: 'Aurora',
    loves: [ 'carrot' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    name: 'Unicrom',
    loves: [ 'energon' ],
    weight: 984,
    gender: 'm',
    vampires: 182
  },
  {
    name: 'Roooooodles',
    loves: [ 'apple' ],
    weight: 575,
    gender: 'm',
    vampires: 99
  },
  {
    name: 'Solnara',
    loves: [ 'apple' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    name: 'Ayna',
    loves: [ 'strawberry' ],
    weight: 733,
    gender: 'f',
    vampires: 40
  },
  {
    name: 'Kenny',
    loves: [ 'grape' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  },
  {
    name: 'Raleigh',
    loves: [ 'apple' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  },
  {
    name: 'Leia',
    loves: [ 'apple' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  },
  {
    name: 'Pilot',
    loves: [ 'apple' ],

```

2.6 Задание 2.3.1

Вывести список самок единорогов весом от полутонны до 700 кг, исключив вывод идентификатора.

```
learn> db.unicorns.find({gender:'f',weight:{$gte:500, $lte:700}},{_id:0})
[
  {
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  },
  {
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]
```

2.7 Задание 2.3.2

Вывести список самцов единорогов весом от полутонны и предпочитающих grape и lemon, исключив вывод идентификатора.

```
learn> db.unicorns.find({gender:'m',weight:{$gte:500},loves:{$all:['grape','lemon']}},{_id:0})
[
  {
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  }
]
```

2.8 Задание 2.3.3

Найти всех единорогов, не имеющих ключ vampires.

```
learn> db.unicorns.find({vampires:{$exists:false}})
[
  {
    _id: ObjectId('696a2d5fc6472676be8ce5ba'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]
```

2.9 Задание 2.3.4

Вывести список упорядоченный список имен самцов единорогов с информацией об их первом предпочтении.

```
learn> db.unicorns.find({gender:'m'},{_id:0,name:1,loves:{$slice:1}}).sort({name:1})
[
  { name: 'Dunx', loves: [ 'grape' ] },
  { name: 'Horny', loves: [ 'carrot' ] },
  { name: 'Kenny', loves: [ 'grape' ] },
  { name: 'Pilot', loves: [ 'apple' ] },
  { name: 'Raleigh', loves: [ 'apple' ] },
  { name: 'Rooooooodles', loves: [ 'apple' ] },
  { name: 'Unicrom', loves: [ 'energon' ] }
]
```

3 Лабораторная работа 6.2 раздел 3

3.1 Задание 3.1.1

- *Создайте коллекцию towns, включающую следующие документы:*

```
{name: "Punxsutawney ",
populatiuon: 6200,
last_sensus: ISODate("2008-01-31"),
famous_for: [""],
mayor: {
  name: "Jim Wehrle"
}}

{name: "New York",
populatiuon: 22200000,
last_sensus: ISODate("2009-07-31"),
famous_for: ["status of liberty", "food"],
mayor: {
  name: "Michael Bloomberg",
  party: "I"}}

{name: "Portland",
populatiuon: 528000,
last_sensus: ISODate("2009-07-20"),
famous_for: ["beer", "food"],
mayor: {
  name: "Sam Adams",
  party: "D"}}
```

```
learn> db.towns.insertMany([
  {name: "Punxsutawney ",populatiuon: 6200,last_sensus: ISODate("2008-01-31"), famous_for: [""],mayor: {name: "Jim Wehrle"}},
  {name: "New York",populatiuon: 22200000,last_sensus: ISODate("2009-07-31"),famous_for: ["status of liberty", "food"],mayor: {name: "Michael Bloomberg",party: "I"}},
  {name: "Portland",populatiuon: 528000,last_sensus: ISODate("2009-07-20"),famous_for: ["beer", "food"],mayor: {name: "Sam Adams",party: "D"}}])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('696a4fbacac3cf9a238ce5b0'),
    '1': ObjectId('696a4fbacac3cf9a238ce5b1'),
    '2': ObjectId('696a4fbacac3cf9a238ce5b2')
  }
}
learn> db.towns.find()
[
  {
    _id: ObjectId('696a4fbacac3cf9a238ce5b0'),
    name: 'Punxsutawney ',
    populatiuon: 6200,
    last_sensus: ISODate('2008-01-31T00:00:00.000Z'),
    famous_for: [ '' ],
    mayor: { name: 'Jim Wehrle' }
  },
  {
    _id: ObjectId('696a4fbacac3cf9a238ce5b1'),
    name: 'New York',
    populatiuon: 22200000,
    last_sensus: ISODate('2009-07-31T00:00:00.000Z'),
    famous_for: [ 'status of liberty', 'food' ],
    mayor: { name: 'Michael Bloomberg', party: 'I' }
  },
  {
    _id: ObjectId('696a4fbacac3cf9a238ce5b2'),
    name: 'Portland',
    populatiuon: 528000,
    last_sensus: ISODate('2009-07-20T00:00:00.000Z'),
    famous_for: [ 'beer', 'food' ],
    mayor: { name: 'Sam Adams', party: 'D' }
  }
]
```


– Сформировать запрос, который возвращает список городов с независимыми мэрами (party="I") . Вывести только название города и информацию о мэре.

```
learn> db.towns.find({'mayor.party':'I'},{name:1,mayor:1,_id:0})
[
  { name: 'New York', mayor: { name: 'Michael Bloomberg', party: 'I' } }
]
```

– Сформировать запрос, который возвращает список беспартийных мэров (party отсутствует) . Вывести только название города и информацию о мэре.

```
learn> db.towns.find({'mayor.party':{'$exists:false'}},{name:1,mayor:1,_id:0})
[ { name: 'Punxsutawney ', mayor: { name: 'Jim Wehrle' } } ]
```

3.2 Задание 3.1.2

- *Сформировать функцию для вывода списка самцов единорогов.*

```
learn> function UnicornsM() {return db.unicorns.find({gender:'m'});}
[Function: UnicornsM]
learn> UnicornsM()
[
  {
    _id: ObjectId('696a4ff2cac3cf9a238ce5b3'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('696a4ff2cac3cf9a238ce5b5'),
    name: 'Unicrom',
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 182
  },
  {
    _id: ObjectId('696a4ff2cac3cf9a238ce5b6'),
    name: 'Rooooooodles',
    loves: [ 'apple' ],
    weight: 575,
    gender: 'm',
    vampires: 99
  },
  {
    _id: ObjectId('696a4ff2cac3cf9a238ce5b9'),
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  },
  {
    _id: ObjectId('696a4ff2cac3cf9a238ce5ba'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  },
  {
    _id: ObjectId('696a4ff2cac3cf9a238ce5bc'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],
    weight: 650,
    gender: 'm',
    vampires: 54
  },
  {
    _id: ObjectId('696a5038cac3cf9a238ce5be'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  }
]
```

– Создать курсор для этого списка из первых двух особей с сортировкой в лексикографическом порядке.

```
learn> var cursor = UnicornsM().limit(2).sort({name:1})
```

– Вывести результат, используя `forEach`.

```
learn> cursor.forEach(function(obj){print(obj);})
{
  _id: ObjectId('696a5038cac3cf9a238ce5be'),
  name: 'Dunx',
  loves: [ 'grape', 'watermelon' ],
  weight: 704,
  gender: 'm',
  vampires: 165
}
{
  _id: ObjectId('696a4ff2cac3cf9a238ce5b3'),
  name: 'Horny',
  loves: [ 'carrot', 'papaya' ],
  weight: 600,
  gender: 'm',
  vampires: 63
}
```

3.3 Задание 3.2.1

Вывести количество самок единорогов весом от полутонны до 600 кг.

```
learn> db.unicorns.find({gender:'f', weight:{$gte:500,$lte:600}}).count()
2
learn>
```

3.4 Задание 3.2.2

Вывести список предпочтений.

```
learn> db.unicorns.distinct('loves')
[
  'apple',      'carrot',
  'chocolate', 'energon',
  'grape',      'lemon',
  'papaya',     'redbull',
  'strawberry', 'sugar',
  'watermelon'
]
```

3.5 Задание 3.2.3

Посчитать количество особей единорогов обоих полов.

```
learn> db.unicorns.aggregate([{$group:{_id:'$gender',count:{$sum:1}}}]])
[ { _id: 'f', count: 5 }, { _id: 'm', count: 7 } ]
```

3.6 Задание 3.3.1

– *Выполнить команду:*

```
> db.unicorns.save({name: 'Barney', loves: ['grape'],
weight: 340, gender: 'm'})
```

```
learn> db.unicorns.save({name: 'Barney', loves: ['grape'], weight: 340, gender: 'm'})
TypeError: db.unicorns.save is not a function
```

– *Проверить содержимое коллекции unicorns.*

Не стал проверять, так как прошлая команда не сработала.

3.7 Задание 3.3.2

– *Для самки единорога Айна внести изменения в БД: теперь ее вес 800, она убила 51 вампира.*

– *Проверить содержимое коллекции unicorns.*

```
learn> db.unicorns.find({name:'Ayna'})
[
  {
    _id: ObjectId('696a4ff2cac3cf9a238ce5b8'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 733,
    gender: 'f',
    vampires: 40
  }
]
learn> db.unicorns.update({gender:'f',name:'Ayna'},{$set:{weight:800,vampires:51}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.find({name:'Ayna'})
[
  {
    _id: ObjectId('696a4ff2cac3cf9a238ce5b8'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 800,
    gender: 'f',
    vampires: 51
  }
]
```

3.8 Задание 3.3.3

- Для самца единорога `Raleigh` внести изменения в БД: теперь он любит рэдбул.
- Проверить содержимое коллекции `unicorns`.

```
learn> db.unicorns.find({name:'Raleigh'})
[
  {
    _id: ObjectId('696a4ff2cac3cf9a238ce5ba'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  }
]
learn> db.unicorns.update({gender:'m',name:'Raleigh'},{$push:{loves:'redbul'}})
DeprecationWarning: Collection.update() is deprecated. Use updateOne, updateMany, or bulkWrite
.
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.find({name:'Raleigh'})
[
  {
    _id: ObjectId('696a4ff2cac3cf9a238ce5ba'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar', 'redbul' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  }
]
```

3.9 Задание 3.3.4

- *Всем самцам единорогов увеличить количество убитых вампиров на 5.*
- *Проверить содержимое коллекции `unicorns`.*

```
learn> db.unicorns.find({gender:'m'},{vampires:1})
[
  { _id: ObjectId('696a4ff2cac3cf9a238ce5b3'), vampires: 63 },
  { _id: ObjectId('696a4ff2cac3cf9a238ce5b5'), vampires: 182 },
  { _id: ObjectId('696a4ff2cac3cf9a238ce5b6'), vampires: 99 },
  { _id: ObjectId('696a4ff2cac3cf9a238ce5b9'), vampires: 39 },
  { _id: ObjectId('696a4ff2cac3cf9a238ce5ba'), vampires: 2 },
  { _id: ObjectId('696a4ff2cac3cf9a238ce5bc'), vampires: 54 },
  { _id: ObjectId('696a5038cac3cf9a238ce5be'), vampires: 165 }
]
learn> db.unicorns.update({gender:'m'},{$inc:{vampires:5}},{multi:true})
Uncaught:
SyntaxError: Unexpected token, expected ",", (1:63)

> 1 | db.unicorns.update({gender:'m'},{$inc:{vampires:5}},{multi:true})
    |                                                                    ^
    2 |

learn> db.unicorns.update({gender:'m'},{$inc:{vampires:5}},{multi:true})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 7,
  modifiedCount: 7,
  upsertedCount: 0
}
learn> db.unicorns.find({gender:'m'},{vampires:1})
[
  { _id: ObjectId('696a4ff2cac3cf9a238ce5b3'), vampires: 68 },
  { _id: ObjectId('696a4ff2cac3cf9a238ce5b5'), vampires: 187 },
  { _id: ObjectId('696a4ff2cac3cf9a238ce5b6'), vampires: 104 },
  { _id: ObjectId('696a4ff2cac3cf9a238ce5b9'), vampires: 44 },
  { _id: ObjectId('696a4ff2cac3cf9a238ce5ba'), vampires: 7 },
  { _id: ObjectId('696a4ff2cac3cf9a238ce5bc'), vampires: 59 },
  { _id: ObjectId('696a5038cac3cf9a238ce5be'), vampires: 170 }
]
```

3.10 Задание 3.3.5

- Изменить информацию о городе Портланд: мэр этого города теперь беспартийный.
- Проверить содержимое коллекции `towns`.

```
learn> db.towns.find({name:'Portland'})
[
  {
    _id: ObjectId('696a4fbacac3cf9a238ce5b2'),
    name: 'Portland',
    populatiuon: 528000,
    last_sensus: ISODate('2009-07-20T00:00:00.000Z'),
    famous_for: [ 'beer', 'food' ],
    mayor: { name: 'Sam Adams', party: 'D' }
  }
]
learn> db.towns.update({name:'Portland'},{$unset:{'mayor.party':1}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.towns.find({name:'Portland'})
[
  {
    _id: ObjectId('696a4fbacac3cf9a238ce5b2'),
    name: 'Portland',
    populatiuon: 528000,
    last_sensus: ISODate('2009-07-20T00:00:00.000Z'),
    famous_for: [ 'beer', 'food' ],
    mayor: { name: 'Sam Adams' }
  }
]
```

3.11 Задание 3.3.6

- Изменить информацию о самце единорога *Pilot*: теперь он любит и шоколад.
- Проверить содержимое коллекции *unicorns*.

```
learn> db.unicorns.find({name:'Pilot'})
[
  {
    _id: ObjectId('696a4ff2cac3cf9a238ce5bc'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],
    weight: 650,
    gender: 'm',
    vampires: 59
  }
]
learn> db.unicorns.update({gender:'m',name:'Pilot'},{$push:{loves:'chocolate'}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.find({name:'Pilot'})
[
  {
    _id: ObjectId('696a4ff2cac3cf9a238ce5bc'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon', 'chocolate' ],
    weight: 650,
    gender: 'm',
    vampires: 59
  }
]
```

3.12 Задание 3.3.7

- Изменить информацию о самке единорога *Aurora*: теперь она любит еще и сахар, и лимоны.
- Проверить содержимое коллекции *unicorns*.

```
learn> db.unicorns.find({name:'Aurora'})
[
  {
    _id: ObjectId('696a4ff2cac3cf9a238ce5b4'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  }
]
learn> db.unicorns.update({gender:'f',name:'Aurora'},{$addToSet:{loves:{$each:['sugar','lemon']}}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.find({name:'Aurora'})
[
  {
    _id: ObjectId('696a4ff2cac3cf9a238ce5b4'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape', 'sugar', 'lemon' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  }
]
```


3.13 Задание 3.4.1

- *Создайте коллекцию towns.*

```
learn> db.towns.find()
[
  {
    _id: ObjectId('696a4fbacac3cf9a238ce5b0'),
    name: 'Punxsutawney ',
    populatiuon: 6200,
    last_sensus: ISODate('2008-01-31T00:00:00.000Z'),
    famous_for: [ '' ],
    mayor: { name: 'Jim Wehrle' }
  },
  {
    _id: ObjectId('696a4fbacac3cf9a238ce5b1'),
    name: 'New York',
    populatiuon: 22200000,
    last_sensus: ISODate('2009-07-31T00:00:00.000Z'),
    famous_for: [ 'status of liberty', 'food' ],
    mayor: { name: 'Michael Bloomberg', party: 'I' }
  },
  {
    _id: ObjectId('696a4fbacac3cf9a238ce5b2'),
    name: 'Portland',
    populatiuon: 528000,
    last_sensus: ISODate('2009-07-20T00:00:00.000Z'),
    famous_for: [ 'beer', 'food' ],
    mayor: { name: 'Sam Adams' }
  }
]
```

- *Удалите документы с беспартийными мэрами.*
- *Проверьте содержание коллекции.*

```
learn> db.towns.delete({'mayor.party':{'$exists:0'}})
TypeError: db.towns.delete is not a function
learn> db.towns.delete
db.towns.deleteMany db.towns.deleteOne

learn> db.towns.delete
db.towns.deleteMany db.towns.deleteOne

learn> db.towns.deleteMany({'mayor.party':{'$exists:0'}})
{ acknowledged: true, deletedCount: 2 }
learn> db.towns.find()
[
  {
    _id: ObjectId('696a4fbacac3cf9a238ce5b1'),
    name: 'New York',
    populatiuon: 22200000,
    last_sensus: ISODate('2009-07-31T00:00:00.000Z'),
    famous_for: [ 'status of liberty', 'food' ],
    mayor: { name: 'Michael Bloomberg', party: 'I' }
  }
]
```

- *Очистите коллекцию.*
- *Просмотрите список доступных коллекций.*

```
learn> db.towns.deleteMany({})  
{ acknowledged: true, deletedCount: 1 }  
learn> db.towns.find()  
  
learn> show collections  
towns  
unicorns
```

4 Лабораторная работа 6.2 раздел 4

4.1 Задание 4.1.1

— *Создайте коллекцию зон обитания единорогов, указав в качестве идентификатора кратко название зоны, далее включив полное название и описание.*

```
learn> db.zones.insertMany([{_id:'forest',name:'Dark forest',description:'Very scary'},{_id:'mountain',name:'Blue mounta
tin',description:'there is a lot of ice and it is slippery'}])
{ acknowledged: true, insertedIds: { '0': 'forest', '1': 'mountain' } }
learn> db.zones.find()
[
  { _id: 'forest', name: 'Dark forest', description: 'Very scary' },
  {
    _id: 'mountain',
    name: 'Blue mountatin',
    description: 'there is a lot of ice and it is slippery'
  }
]
```

— *Включите для нескольких единорогов в документы ссылку на зону обитания, используя второй способ автоматического связывания.*

```
learn> db.unicorns.find({name:'Raleigh'})
[
  {
    _id: ObjectId('696a4ff2cac3cf9a238ce5ba'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar', 'redbul' ],
    weight: 421,
    gender: 'm',
    vampires: 7
  }
]
learn> db.unicorns.update({name:'Raleigh'},{$set:{zone:{$ref:'zones',$id:'mountain'}}})
DeprecationWarning: Collection.update() is deprecated. Use updateOne, updateMany, or bulkWrite.
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.find({name:'Raleigh'})
[
  {
    _id: ObjectId('696a4ff2cac3cf9a238ce5ba'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar', 'redbul' ],
    weight: 421,
    gender: 'm',
    vampires: 7,
    zone: DBRef('zones', 'mountain')
  }
]
learn> db.unicorns.find({name:'Kenny'})
[
  {
    _id: ObjectId('696a4ff2cac3cf9a238ce5b9'),
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 44
  }
]
```

Продолжение скриншота.

```
learn> db.unicorns.update({name: 'Kenny'}, {$set: {zone: {$ref: 'zones', $id: 'forest'}}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.find({name: 'Kenny'})
[
  {
    _id: ObjectId('696a4ff2cac3cf9a238ce5b9'),
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 44,
    zone: DBRef('zones', 'forest')
  }
]
```

— Проверьте содержание коллекции единорогов.

```
learn> db.unicorns.find()
[
  {
    _id: ObjectId('696a4ff2cac3cf9a238ce5b3'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 68
  },
  {
    _id: ObjectId('696a4ff2cac3cf9a238ce5b4'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape', 'sugar', 'lemon' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('696a4ff2cac3cf9a238ce5b5'),
    name: 'Unicrom',
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 187
  },
  {
    _id: ObjectId('696a4ff2cac3cf9a238ce5b6'),
    name: 'Rooooooodles',
    loves: [ 'apple' ],
    weight: 575,
    gender: 'm',
    vampires: 104
  },
  {
    _id: ObjectId('696a4ff2cac3cf9a238ce5b7'),
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
]
```

```
{
  _id: ObjectId('696a4ff2cac3cf9a238ce5b8'),
  name: 'Ayna',
  loves: [ 'strawberry', 'lemon' ],
  weight: 800,
  gender: 'f',
  vampires: 51
},
{
  _id: ObjectId('696a4ff2cac3cf9a238ce5b9'),
  name: 'Kenny',
  loves: [ 'grape', 'lemon' ],
  weight: 690,
  gender: 'm',
  vampires: 44,
  zone: DBRef('zones', 'forest')
},
{
  _id: ObjectId('696a4ff2cac3cf9a238ce5ba'),
  name: 'Raleigh',
  loves: [ 'apple', 'sugar', 'redbul' ],
  weight: 421,
  gender: 'm',
  vampires: 7,
  zone: DBRef('zones', 'mountain')
},
{
  _id: ObjectId('696a4ff2cac3cf9a238ce5bb'),
  name: 'Leia',
  loves: [ 'apple', 'watermelon' ],
  weight: 601,
  gender: 'f',
  vampires: 33
},
{
  _id: ObjectId('696a4ff2cac3cf9a238ce5bc'),
  name: 'Pilot',
  loves: [ 'apple', 'watermelon', 'chocolate' ],
  weight: 650,
  gender: 'm',
  vampires: 59
}
```

4.2 Задание 4.2.1

Проверьте, можно ли задать для коллекции `unicorns` индекс для ключа `name` с флагом `unique`.

```
learn> db.unicorns.ensureIndex({name:1},{unique:1})
[ 'name_1' ]
learn> |
```

4.3 Задание 4.3.1

- Получите информацию о всех индексах коллекции `unicorns`.
- Удалите все индексы, кроме индекса для идентификатора.
- Попробуйте удалить индекс для идентификатора.

```
learn> db.unicorns.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { name: 1 }, name: 'name_1', unique: true }
]
learn> db.unicorns.dropIndex('name_1')
{ nIndexesWas: 2, ok: 1 }
learn> db.unicorns.getIndexes()
[ { v: 2, key: { _id: 1 }, name: '_id_' } ]
learn> db.unicorns.dropIndex('_id_')
MongoServerError[InvalidOptions]: cannot drop _id index
learn> db.unicorns.getIndexes()
[ { v: 2, key: { _id: 1 }, name: '_id_' } ]
learn> |
```

4.4 Задание 4.4.1

- Создайте объемную коллекцию `numbers`, задействовав курсор:
`for(i = 0; i < 100000; i++){db.numbers.insert({value: i})}`
- Выберите последних четыре документа.

```
learn> show collections
towns
unicorns
zones
learn> for(i = 0; i < 100000; i++){db.numbers.insert({value: i})}
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('696b7ab26c6a0c260e906a32') }
}
learn> db.numbers.find().limit(4).sort({_id:-1})
[
  { _id: ObjectId('696b7ab26c6a0c260e906a32'), value: 99999 },
  { _id: ObjectId('696b7ab26c6a0c260e906a31'), value: 99998 },
  { _id: ObjectId('696b7ab26c6a0c260e906a30'), value: 99997 },
  { _id: ObjectId('696b7ab26c6a0c260e906a2f'), value: 99996 }
]
```

– Проанализируйте план выполнения запроса 2. Сколько потребовалось времени на выполнение запроса? (по значению параметра `executionTimeMillis`)

```
learn> db.numbers.explain("executionStats").find().limit(4).sort({_id:-1})
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'learn.numbers',
    indexFilterSet: false,
    parsedQuery: {},
    queryHash: 'DA838021',
    planCacheKey: 'DA838021',
    optimizationTimeMillis: 1,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    winningPlan: {
      stage: 'LIMIT',
      limitAmount: 4,
      inputStage: {
        stage: 'FETCH',
        inputStage: {
          stage: 'IXSCAN',
          keyPattern: { _id: 1 },
          indexName: '_id_',
          isMultiKey: false,
          isUnique: true,
          isSparse: false,
          isPartial: false,
          indexVersion: 2,
          direction: 'backward',
          indexBounds: { _id: [ '[MaxKey, MinKey]' ] }
        }
      }
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 4,
    executionTimeMillis: 1,
    totalKeysExamined: 4,
    totalDocsExamined: 4,
    executionStages: {
      stage: 'LIMIT',
```

Время выполнения мало (1 миллисекунда). Предполагаю, что это возникло из-за встроенной индексации идентификатора, надо попробовать с

значением (value). Также выведу список индексов для этой коллекции, чтобы убедиться.

```
learn> db.numbers.getIndexes()
[ { v: 2, key: { _id: 1 }, name: '_id_' } ]
learn> db.numbers.explain("executionStats").find().sort({value:-1}).limit(4)
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'learn.numbers',
    indexFilterSet: false,
    parsedQuery: {},
    queryHash: '0DB68434',
    planCacheKey: '0DB68434',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    winningPlan: {
      stage: 'SORT',
      sortPattern: { value: -1 },
      memLimit: 104857600,
      limitAmount: 4,
      type: 'simple',
      inputStage: { stage: 'COLLSCAN', direction: 'forward' }
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 4,
    executionTimeMillis: 82,
    totalKeysExamined: 0,
    totalDocsExamined: 100000,
    executionStages: {
      stage: 'SORT',
      nReturned: 4,
      executionTimeMillisEstimate: 8,
      works: 100006,
      advanced: 4,
      needTime: 100001,
      needYield: 0,
```

- *Создайте индекс для ключа value.*
- *Получите информацию о всех индексах коллекции numbers.*
- *Выполните запрос 2.*

```

learn> db.numbers.ensureIndex({value:1})
[ 'value_1' ]
learn> db.numbers.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { value: 1 }, name: 'value_1' }
]
learn> db.numbers.explain("executionStats").find().sort({value:-1}).limit(4)
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'learn.numbers',
    indexFilterSet: false,
    parsedQuery: {},
    queryHash: '0DB68434',
    planCacheKey: '0DB68434',
    optimizationTimeMillis: 1,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    winningPlan: {
      stage: 'LIMIT',
      limitAmount: 4,
      inputStage: {
        stage: 'FETCH',
        inputStage: {
          stage: 'IXSCAN',
          keyPattern: { value: 1 },
          indexName: 'value_1',
          isMultiKey: false,
          multiKeyPaths: { value: [] },
          isUnique: false,
          isSparse: false,
          isPartial: false,
          indexVersion: 2,
          direction: 'backward',
          indexBounds: { value: [ '[MaxKey, MinKey]' ] }
        }
      }
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 4,
    executionTimeMillis: 4,
    totalKeysExamined: 4,
    totalDocsExamined: 4,
    executionStages: {
      stage: 'LIMIT',
      nReturned: 4,

```

– Проанализируйте план выполнения запроса с установленным индексом. Сколько потребовалось времени на выполнение запроса?

– Сравните время выполнения запросов с индексом и без. Дайте ответ на вопрос: какой запрос более эффективен?

До индексации время выполнения было 82 миллисекунды, а после 4. Индексация сработала, удалось оптимизировать запрос.

Выводы: в ходе выполнения лабораторной работы были изучены основные возможности базы данных MongoDB, получены навыки работы с коллекциями.